

**NAME:**KRISHNAPRIYA P

**REG NO:**192511279

**Course code:**CSA0613

**Course title:** Design and Analysis of Algorithms

## **CO2\_ASSESSMENT TOOL 3 \_ DEBUGGING & OPTIMISATION TASK**

### **Title: Binary Search – First Element Detection**

#### **Aim**

To implement the Binary Search algorithm to locate the first element (3) in the sorted dataset [3, 8, 14, 19, 25, 31, 40], count the number of comparisons required, and analyze its time complexity.

#### **Software Used**

- Python

#### **Algorithm**

- Start.
- Read the sorted array and the target element.
- Initialize low = 0 and high = n - 1.
- Repeat while low <= high.
- Find the middle index using  $\text{mid} = (\text{low} + \text{high}) // 2$ .
- Compare the middle element with the target.
- If the middle element equals the target, display the index and stop.
- If the target is smaller than the middle element, update high = mid - 1.
- Otherwise, update low = mid + 1.
- Display the total number of comparisons.
- Stop.

## Source Code (Python)

```
main.py    Share  Run

1  arr = [3, 8, 14, 19, 25, 31, 40]
2  target = 3
3  low = 0
4  high = len(arr) - 1
5  comparisons = 0
6  while low <= high:
7      mid = (low + high) // 2
8      comparisons += 1
9      print("Comparison", comparisons, ": Checking", arr[mid])
10     if arr[mid] == target:
11         print("\nElement Found")
12         print("Index:", mid)
13         break
14     elif target < arr[mid]:
15         high = mid - 1
16     else:
17         low = mid + 1
18 else:
19     print("Element Not Found")
```

## OUTPUT

```
Output

Comparison 1 : Checking 19
Comparison 2 : Checking 8
Comparison 3 : Checking 3

Element Found
Index: 0
Total Comparisons: 3

=== Code Execution Successful ===
```

## Analysis

### Problem Interpretation

- The given dataset is [3, 8, 14, 19, 25, 31, 40], which is already sorted.
- The objective is to find the element 3 using Binary Search.
- Since Binary Search works efficiently on sorted data, it is the most suitable algorithm for this problem.

### Search Process

- **Step 1:** Middle element = 19, target is smaller → search left half.
- **Step 2:** Middle element = 8, target is smaller → search left half.
- **Step 3:** Middle element = 3, target found at **index 0**.

### Documentation

- Binary Search repeatedly divides the search interval into two halves.
- Even though the target is the **first element**, the algorithm first checks the middle element before narrowing the search.
- The target element 3 is found after **3 comparisons**.
- The algorithm correctly returns **index 0**, verifying the correctness of the implementation.
- **Time Complexity:**
  - Best Case:  $\Theta(1)$
  - Average Case:  $\Theta(\log n)$
  - Worst Case:  $\Theta(\log n)$
- **Space Complexity:**  $\Theta(1)$

